Connected Greenhouse Monitoring

BTS academic project. Public documentation extract covering the system architecture, sensor processing, displays, database structure and Sigfox communication. Connection examples and full source-code appendices are omitted. Original report page numbers are retained.

Portfolio documentation | Iakobi Iakobashvili

Cahier des charges

Diagramme de déploiement

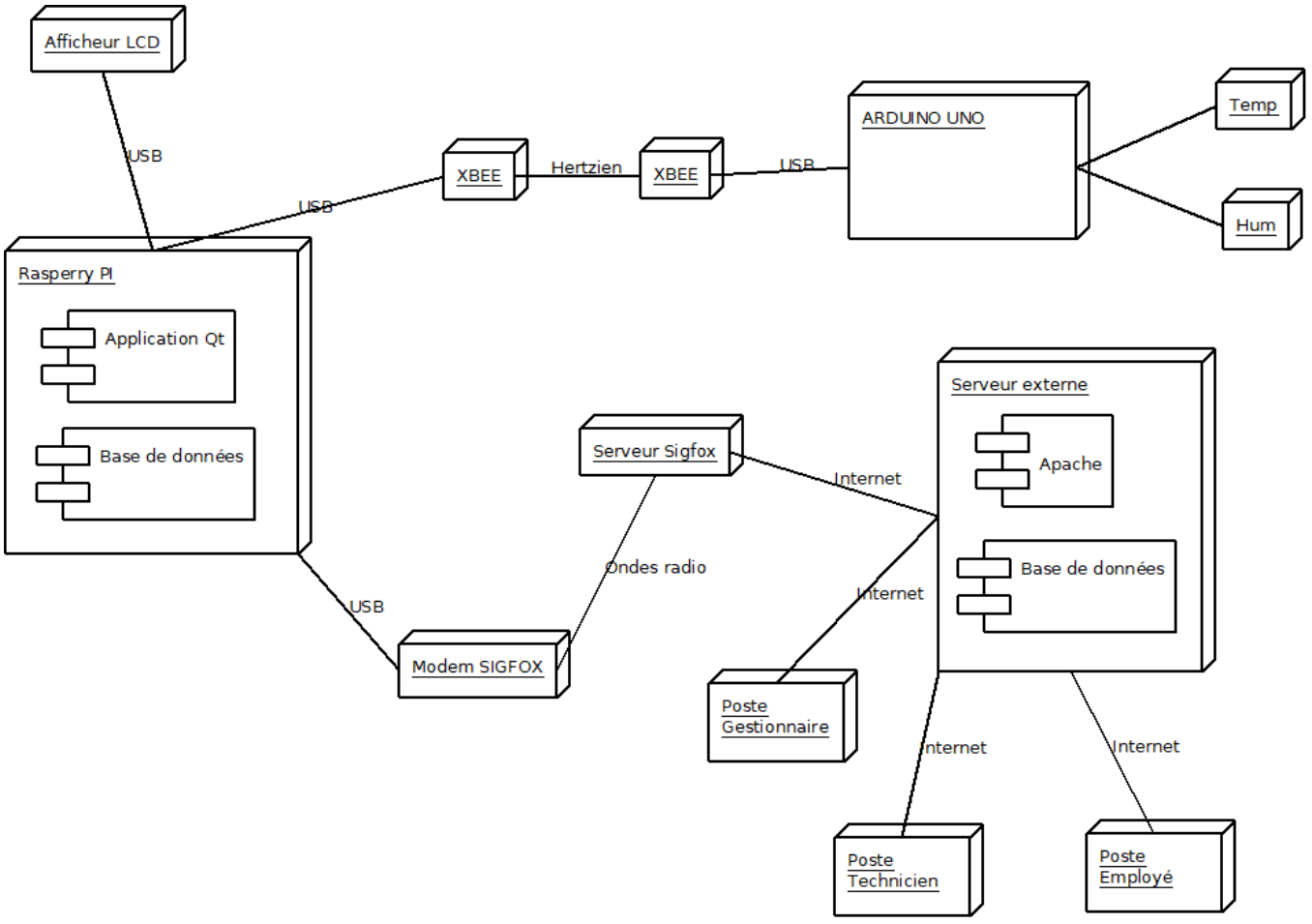
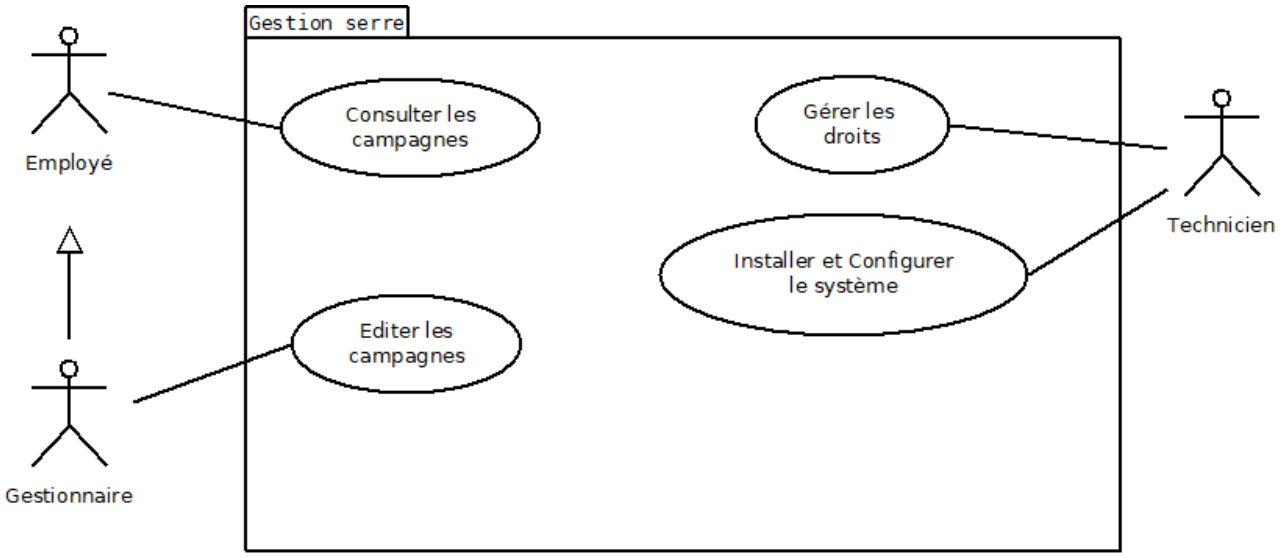


Diagramme des cas d’utilisation

Le diagramme ci-dessous illustre les interactions possibles entre les différents types d'utilisateurs (acteur) et les fonctionnalités (cas d'utilisation) du système de gestion de serre.



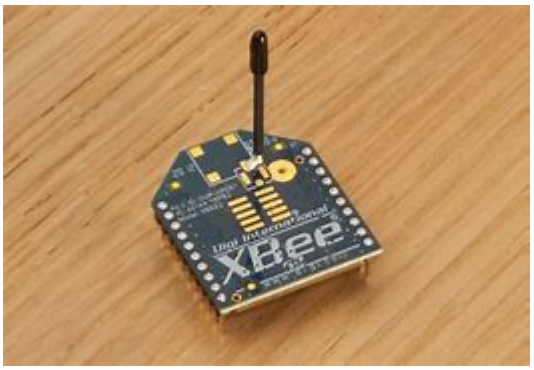
Explication des rôles :

- **Employé** : a uniquement accès à la consultation des campagnes (affichage des données environnementales).
- **Gestionnaire** : hérite des droits de l'employé et peut en plus éditer les campagnes, c’est-à-dire gérer les cultures (ajout, modification, suppression) et leurs périodes.
- **Technicien** : peut gérer les droits d'accès (création, modification et suppression de comptes utilisateurs), et intervenir dans l'installation et la configuration du système.

Ce diagramme UML permet de bien structurer les responsabilités de chaque utilisateur dans le projet, et facilite la compréhension du fonctionnement général du système.

Éléments à disposition

XBEE récepteur



- **Alimentation** : 3.3V
- **Fréquence** : 2.4 GHz
- **Antenne** : unipolaire U.FL
- **Connexion** : sans fil (RF)
- **Portée** : jusqu'à 90 mètres en intérieur, 1.6 km en extérieur (selon le modèle)
- **Interface de communication** : UART, SPI

Raspberry Pi



- **Mémoire RAM** : 1 Go (variable selon le modèle)
- **Système d'exploitation** : Raspberry Pi OS (ou autre OS compatible)
- **Connectivité** : Wi-Fi, Bluetooth, GPIO, USB, HDMI
- **Alimentation** : 5V / 2.5A via micro-USB ou USB-C

Afficheur LCD programmable



Taille	70cm x 11cm
Leds	560 leds organisées en 80 colonnes de 7 leds (61cm x 5.5cm)
Couleur des leds	monochrome (rouge) ou 3 couleurs (rouge, vert et orange)
Luminosité	Réglable sur 4 niveaux : 25%, 50%, 75%, 100%
Affichage	Texte et graphiques, fixes et défilants, 16 graphiques personnalisables
Polices	3 polices :étroite, normale, gras
Effets	Effets d'apparition et de disparition, affichage fixe et clignotant, beep, ...
Gestion de l'heure	Horloge réglable sauvegardée, possibilité d'affichage de la date et de l'heure
Capacité	26 pages de 155 caractères
Programmation horaire	5 programmations différentes de 31 pages (année, mois, jour, heure, minute)
Liaison ordinateur	Port série RS232 (connecteur SubD9) ou USB (port série virtuel)

Les paramètres de la communication:

• Vitesse : 9600 bauds

• Bits de données : 8

• Bit de parité : aucun

• Bit de stop : 1

• Contrôle de flux : aucun

Modem Sigfox



- **Bande de fréquence** : 868 MHz (Europe) / 902 MHz (Amérique)
- **Portée** : Jusqu'à 50 km en zone rurale, 10 km en zone urbaine
- **Débit** : 100 bits/s
- **Consommation énergétique** : Très faible, optimisé pour les applications IoT
- **Interface de communication** : UART / AT Commands

Élément	Rôle
XBEE	Réception sans fil des trames (UART)
Raspberry Pi 3	Exécution du programme Qt, stockage local
Afficheur LED	Affichage défilant de la température et humidité
Modem Sigfox	Transmission distante des données
Qt Creator	Environnement de développement
MySQL + PHP	Stockage + consultation des données

Fonctionnements

Architecture du programme

- Lecture série des trames via le port COM.
- Extraction des valeurs (n° serre, température x10, humidité).
- Affichage dans une IHM Qt en temps réel.
- Sauvegarde automatique toutes les minutes.
- Envoi vers LED par liaison RS232.
- Envoi vers Sigfox via commandes AT (mode commande).

Fonctionnalités principales

- Interface Qt avec boutons d'action et trame log.
- Affichage dynamique dans le widget texte.
- Connexion MySQL à `serre_data`.
- Affichage LED avec effets visuels.
- Transmission vers Sigfox au format hexadécimal.

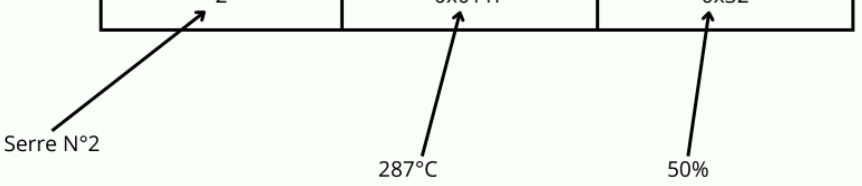
Réception XBEE

- Lecture UART (9600 bauds) via `QSerialPort`
- Thread dédié pour éviter de bloquer l'interface
- Trames de 4 octets (serre, température PF | température pf, humidité)
  La valeur de température multipliée par 10 pour recevoir la valeur décimale non pointée

Format de données

- 4 octets envoyés

N° Serre	Température *10 pour avoir une valeur entière	Humidité %
1 Octet	2 Octets PF pf	1 Octet
2	0x011F	0x32



Préparation du port série pour la réception

```
void Widget::recevoir()
{
    if (ouvrirPort(serial, "COM11", QSerialPort::Baud9600))
    {
        connect(serial, &QSerialPort::readyRead, this, &Widget::readData);
        ui->trame->append("  Réception en cours...");
    }

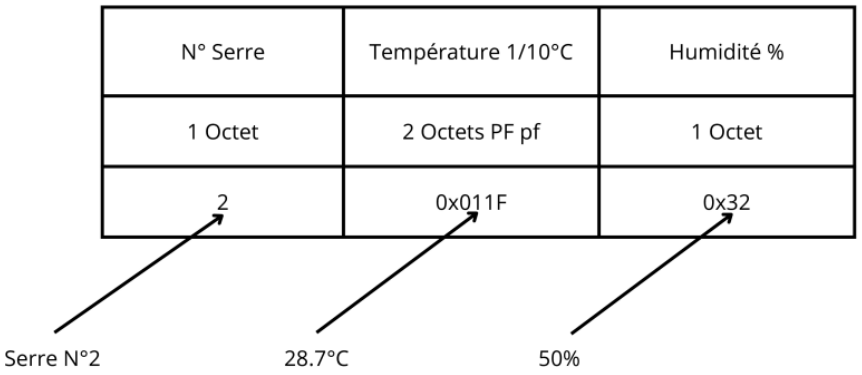
}
```

Traitement des données

- Numéro serre: octet 1
- Calcul de la température octet 2 et 3: `la valeur est divisée par 10 pour retrouver la valeur précise décimale pointée`
- Humidité: octet 4
- Stockage dans des variables internes Qt

Format de données

- 4 octets reçus



Traîtement de données

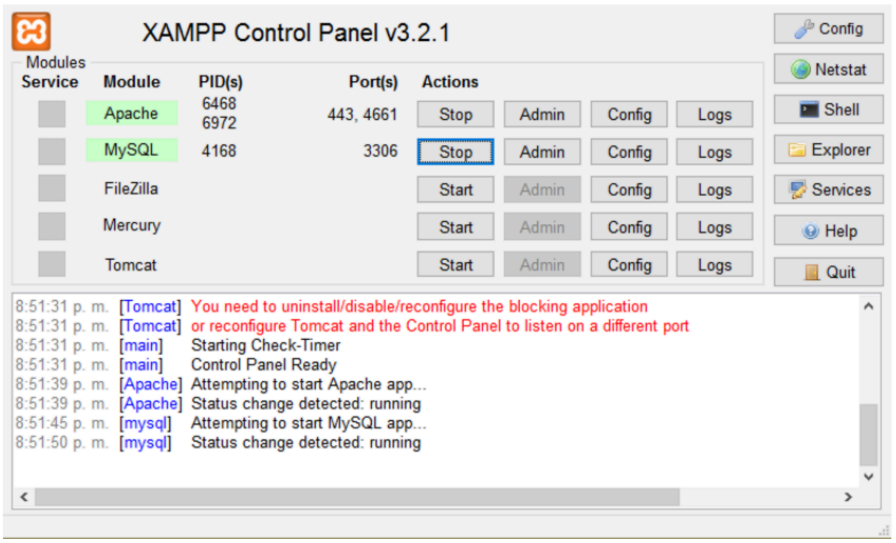
```
// Lecture des données reçues
void Widget::readData()
{
    dataBuffer.append(serial->readAll());

    while (dataBuffer.size() >= 4)
    {
        QByteArray trame = dataBuffer.left(4);
        dataBuffer.remove(0, 4);

        // Chaque réception écrase les anciennes valeurs
        numeroSerre = static_cast<quint8>(trame[0]);
        temperatureSerre = (static_cast<qint16>((static_cast<quint8>(trame[1]) << 8) | static_cast<quint8>(trame[2]))) / 10.0;
        humiditeSerre = static_cast<quint8>(trame[3]);
```

Base de données locale

- Serveur MySQL via XAMPP



- Création BDD serre_data, Table `capteurs` : “ID, numero_serre, temperature, humidite, date_maj”
- Dans l’onglet “SQL” de phpMyAdmin écrire:

Requête SQL

```sql
CREATE TABLE capteurs (
    id INT AUTO_INCREMENT PRIMARY KEY,
    numero_serre INT NOT NULL,
    temperature DECIMAL(5,2) NOT NULL,
    humidite INT NOT NULL,
    date_maj TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Affichage local IHM

- QtWidget avec zones de texte dynamiques
- Rafraîchissement à chaque trame reçue

Affichage des octets organisés avec du texte devant

```
// Afficher les nouvelles valeurs
QString affichage = QString("\nSerre %1\nTempérature : %2 °C\nHumidité : %3 %\n")
                        .arg(numeroSerre)
                        .arg(temperatureSerre)
                        .arg(humiditeSerre);

ui->trame->append(affichage);
```

Affichage LED

- Trame RS232 : format spécifique avec checksum
- Exemple : <ID01><L1><PA>...<E>
- Exemple de message défilant : "Serre 1 - Température : 28.7°C - Humidité : 65%"

Syntaxe intégrée

```
QString sensorMessage = QString("Serre %1\nTempérature : %2 °C\nHumidité : %3 %\n")
                            .arg(numeroSerre)
                            .arg(temperatureSerre)
                            .arg(humiditeSerre);

QString dataField = "<L1><PA><FE><MQ><WC><FA:   " + sensorMessage;
QString trame = "<ID01>" + dataField + computeChecksum(dataField) + "<E>\r\n";
```

Transmission vers Sigfox

- Conversion des valeurs en hexadécimal (payload)
- Envoi AT$SF=...
- Attente de réponse et gestion d'erreur
- Envoi de données programmée toutes les 11 ou 12 minutes, car:

  Limite messages SigFox: 6 messages de 12 octets par heure ou 140 messages par jour

  Toutes les 10 minutes: 60 / 10 x 24 = 144 messages > 140 messages par jour

  Toutes les 11 minutes: 60 / 11 x 24 = entre 130 et 131 messages < 140 messages par jour

  Toutes les 12 minutes: 60 / 12 x 24 = 120 messages < 140 messages par jour

Vérification du mode AT

```
if (!ouvrirPort(sigfoxSerial, "COM9", QSerialPort::Baud19200)) return;

// 1. Tester si le modem répond à "AT\r\n"
sigfoxSerial->write("AT\r\n");
if (!sigfoxSerial->waitForReadyRead(1000))
{
    // Pas de réponse -> envoyer +++
    sigfoxSerial->write("+++");
    sigfoxSerial->flush();
    if (!sigfoxSerial->waitForReadyRead(5000))
    {
        ui->trame->append("❌ Pas de réponse au passage en mode commande.");
        sigfoxSerial->close();
        return;
    }
    ui->trame->append("✅ Mode commande activé.");
} else {
    ui->trame->append("✅ Modem déjà en mode commande.");
}
```

Préparation et l’envois des commandes en mode AT

```
QString hexPayload = payload.toHex().toUpper();
QString atCommand = "AT$SF=" + hexPayload + ",1\r\n";

sigfoxSerial->write(atCommand.toUtf8());
sigfoxSerial->flush();

if (sigfoxSerial->waitForReadyRead(7000))
    ui->trame->append("🔄 Réponse Sigfox : " + QString(sigfoxSerial->readAll()));
else
    ui->trame->append("⏳ Aucune réponse du modem Sigfox.");

sigfoxSerial->close();
```

Programmer l’envoi toutes les 11 minutes

```
void Widget::BT_envoie_sigfox()
{
    envoyerSigfox();
    sigfoxTimer->start(11 * 60 * 1000); // Toutes les 11 minutes
    ui->trame->append("⏳ Envoi Sigfox programmé toutes les 11 minutes.");

}
```

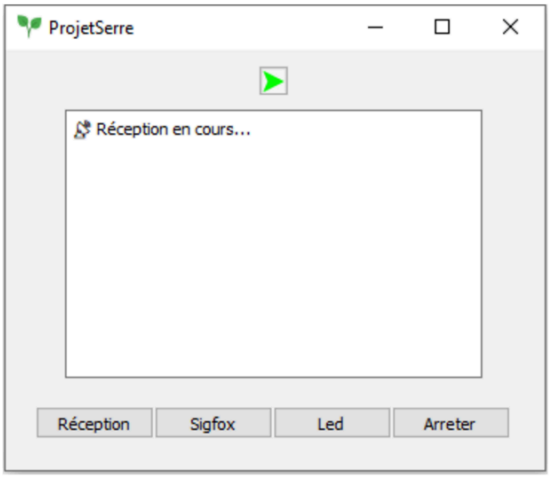
Manipulations:

Préparation du port série pour la réception

```
void Widget::recevoir()
{
    if (ouvrirPort(serial, "COM11", QSerialPort::Baud9600))
    {
        connect(serial, &QSerialPort::readyRead, this, &Widget::readData);
        ui->trame->append(" Réception en cours...");
    }

}
```

Résultat

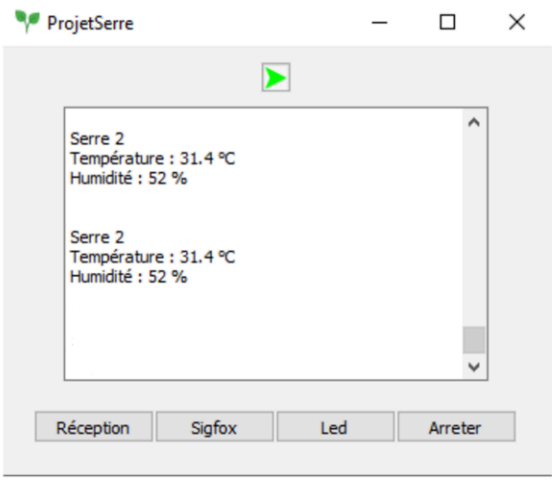


Affichage des octets organisés avec du texte devant

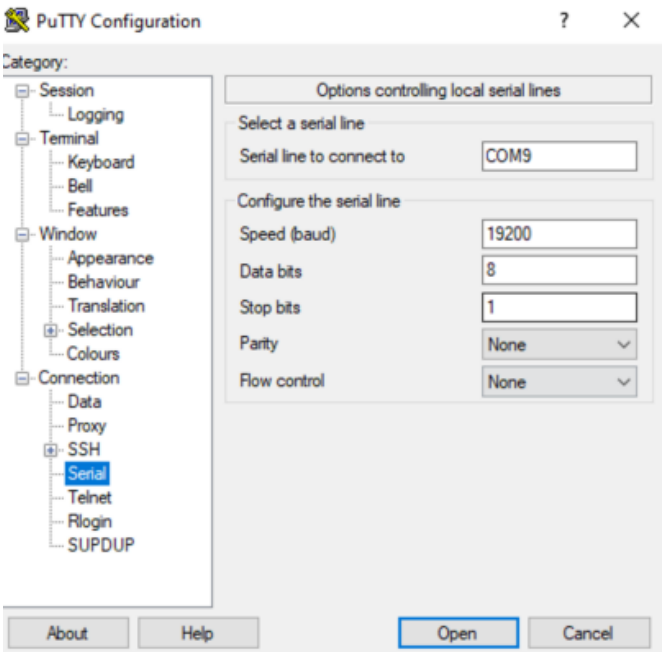
```
// Afficher les nouvelles valeurs
QString affichage = QString("\nSerre %1\nTempérature : %2 °C\nHumidité : %3 %\n")
                        .arg(numeroSerre)
                        .arg(temperatureSerre)
                        .arg(humiditeSerre);

ui->trame->append(affichage);
```

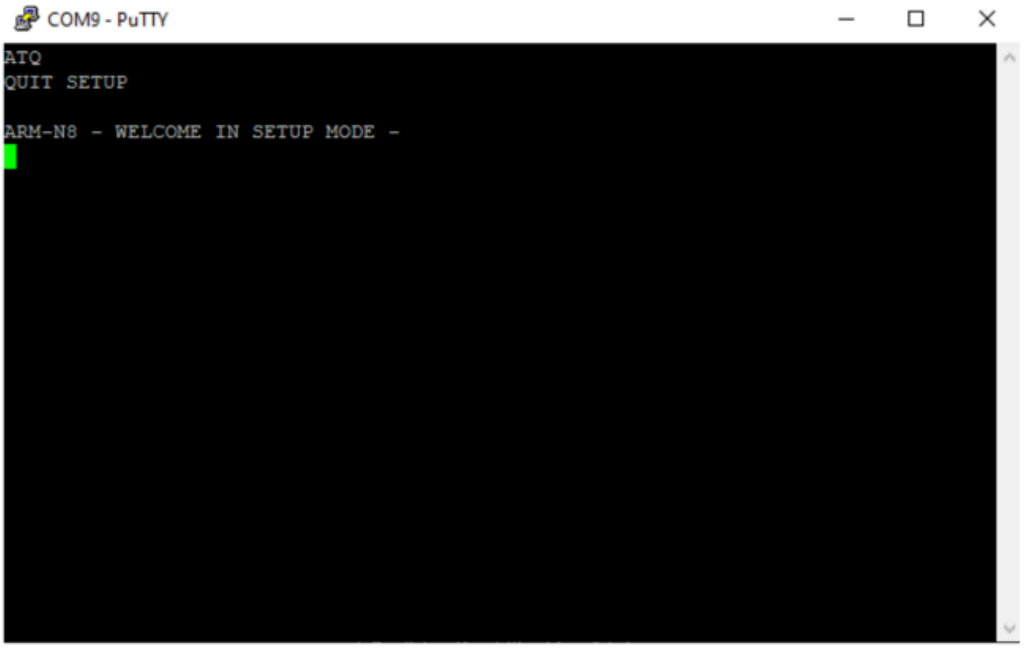
Résultat



Configuration Modem Sigfox dans PuTTY



Mode commande (AT)
écrire: “+++”



Au départ, le modem Sigfox est configuré par défaut en **mode transparent**.
Dans ce mode, tout ce qui est envoyé est transmis tel quel, sans interprétation.

Pour envoyer des commandes spécifiques, il est nécessaire de passer en **mode commande**, où chaque instruction doit commencer par le préfixe syntaxique `AT$SF`, suivi de la commande souhaitée.

Le choix du **mode commande (AT)** a été fait car il permet de recevoir une réponse du modem. Cela permet de **vérifier l'état de la commande** à sa réception et de s'assurer que **les données ont bien été transmises**, grâce à un retour d'information (accusé de réception ou message d'erreur).

En C++:

Vérification du mode AT

```
if (!ouvrirPort(sigfoxSerial, "COM9", QSerialPort::Baud19200)) return;

// 1. Tester si le modem répond à "AT\r\n"
sigfoxSerial->write("AT\r\n");
if (!sigfoxSerial->waitForReadyRead(1000))
{
    // Pas de réponse -> envoyer +++
    sigfoxSerial->write("+++");
    sigfoxSerial->flush();
    if (!sigfoxSerial->waitForReadyRead(5000))
    {
        ui->trame->append("❌ Pas de réponse au passage en mode commande.");
        sigfoxSerial->close();
        return;
    }
    ui->trame->append("✅ Mode commande activé.");
} else {
    ui->trame->append("✅ Modem déjà en mode commande.");
}
```

Préparation et l'envois des commandes en mode AT

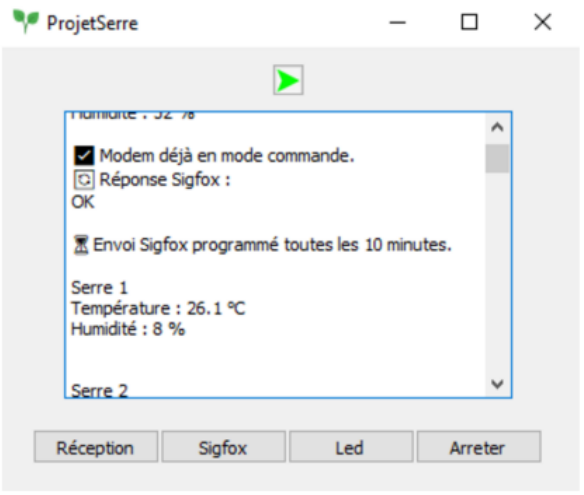
```
QString hexPayload = payload.toHex().toUpper();
QString atCommand = "AT$SF=" + hexPayload + ",1\r\n";

sigfoxSerial->write(atCommand.toUtf8());
sigfoxSerial->flush();

if (sigfoxSerial->waitForReadyRead(7000))
    ui->trame->append("🔄 Réponse Sigfox : " + QString(sigfoxSerial->readAll()));
else
    ui->trame->append("⏳ Aucune réponse du modem Sigfox.");

sigfoxSerial->close();
```

Résultat

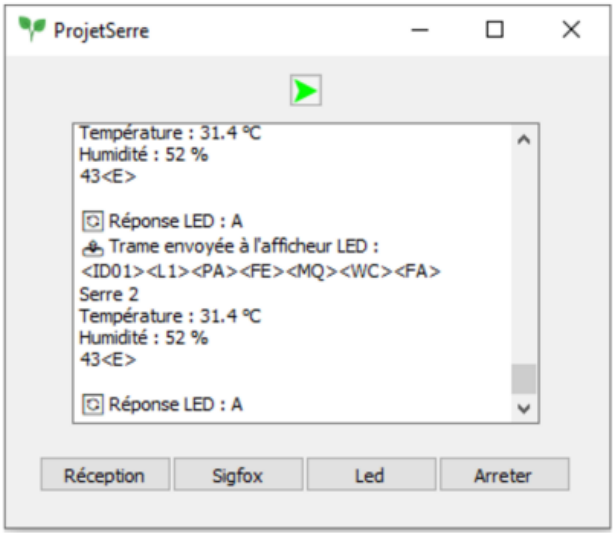


Affichage de données sur le LED

```
QString sensorMessage = QString("Serre %1\nTempérature : %2 °C\nHumidité : %3 %\n")
                        .arg(numeroSerre)
                        .arg(temperatureSerre)
                        .arg(humiditeSerre);

QString dataField = "<L1><PA><FE><MQ><WC><FA>\n" + sensorMessage;
QString trame = "<ID01>" + dataField + computeChecksum(dataField) + "<E>\r\n";
```

Résultat



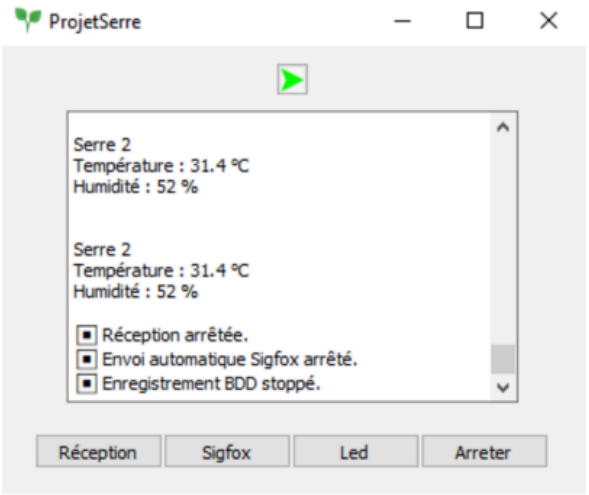
Résultat



Arrêter tout (bouton “Arrêter”)

```
// Fonction pour arrêter la communication
void Widget::arreter()
{
    serial->close();
    ledSerial->close();
    sigfoxSerial->close();
    dataBuffer.clear();
}
```

Résultat



La structure de base de données

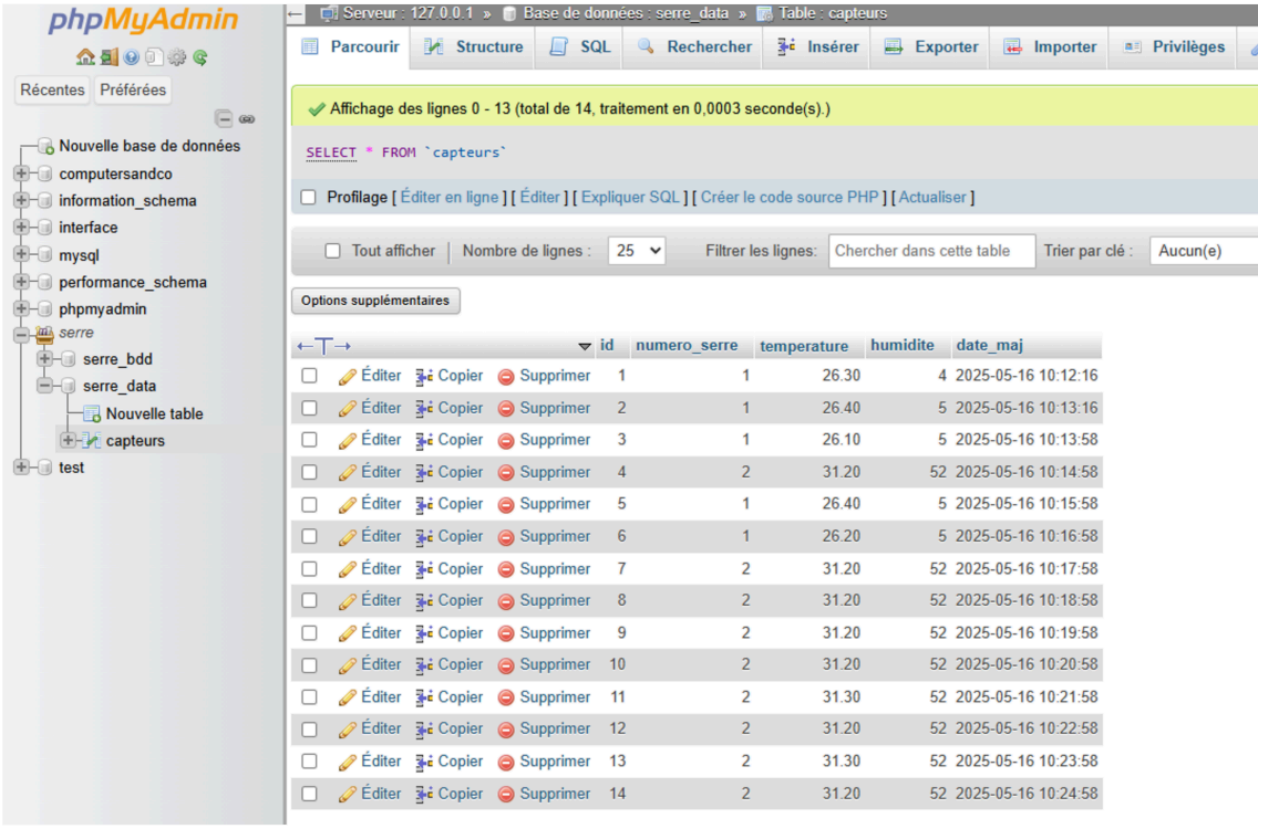


Tableau relationnel de la table “capteurs”

Nom du champ	Type de données	Clé	Contraintes	Description
`id`	INT	🔑 PK	AUTO_INCREMENT, NOT NULL	Identifiant unique de chaque enregistrement
`numero_serre`	INT		NOT NULL	Numéro de la serre
`temperature`	FLOAT		NOT NULL	Température mesurée
`humidite`	INT		NOT NULL	Humidité mesurée (%)
`date_maj`	DATETIME		DEFAULT CURRENT_TIMESTAMP	Date/heure d'enregistrement